

Supply Chain Analytics Using AI

Supply Chain Analytics for AtliQ Mart using N8N, Postgres & Quadratic

1. Problem Statement

AtliQ Mart is a Gujarat-based organic food manufacturer. It specialises in only a few products but has disrupted the market in the two cities where it operates. In an interesting move, it opened its third city of operation in New Jersey, USA, and started performing well there too. Although it handles very few products, its supply chain is immature.

Bruce Hurali, the Chief Operating Officer, has noticed that some supermarket customers are dissatisfied with order management, and he wants to fix this before scaling further. He discussed it with **Tony Sharma**, Head of Analytics, and they concluded it is a classic supply chain issue: the company is not able to maintain an optimum inventory.

Tony offered to build a quick insights report in Excel or Power BI, but Bruce insisted on an AI-powered solution. Tony assigned the work to **Peter Pande**, a curious data analyst who explores AI tools. Peter found **Quadratic**, an AI-powered spreadsheet that can connect directly to a Postgres database, and built an automation with **N8N** that migrates data straight from emails into Postgres.

Goal: solve the supply chain problem using Quadratic (AI spreadsheet) and N8N (automated data migration), while learning the key supply chain domain metrics.

2. Architecture of the Stack

- Data arrives through emails (it could come from other sources, but here it comes in by email).
- It is fed into **N8N**, an AI agentic automation tool.
- N8N loads it into **Postgres**, a database management system.
- The data is then pulled into **Quadratic**, an AI spreadsheet, where prompt-based analysis is done.

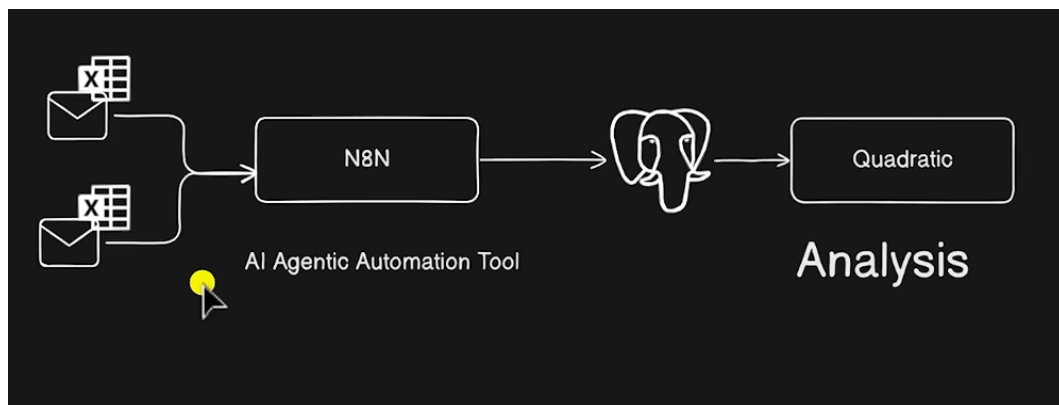


Figure 1: Email → N8N → Postgres → Quadratic (analysis)

With this setup, the N8N agent monitors the inbox, detects new updates and runs the automated workflow.

3. Automated Data Migration Using N8N

1. Go to N8N > **Create a new workflow**.
2. Click **Add first step**. Since the data is fetched from Gmail, choose **Gmail** as the first step.

- Under Triggers, click “On Message Received”.

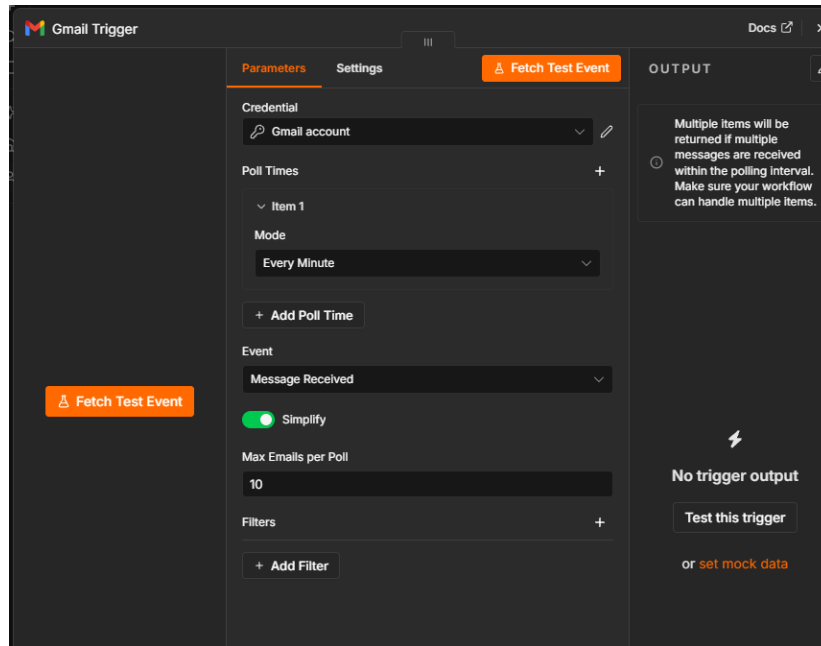


Figure 2: Gmail Trigger configuration

- Configure the trigger and sign in to the Gmail account the data will flow from.
- Set the poll interval (every minute, hour, day, etc.) as you prefer.
- Under **Filters**, choose **Label Names or IDs** > **INBOX**, since that is where the data arrives.
- Add a new filter, **Search**, and enter the common subject line used in the emails carrying the data (e.g. subject:daily sales).
- Under **Options**, switch on **Download Attachments**.

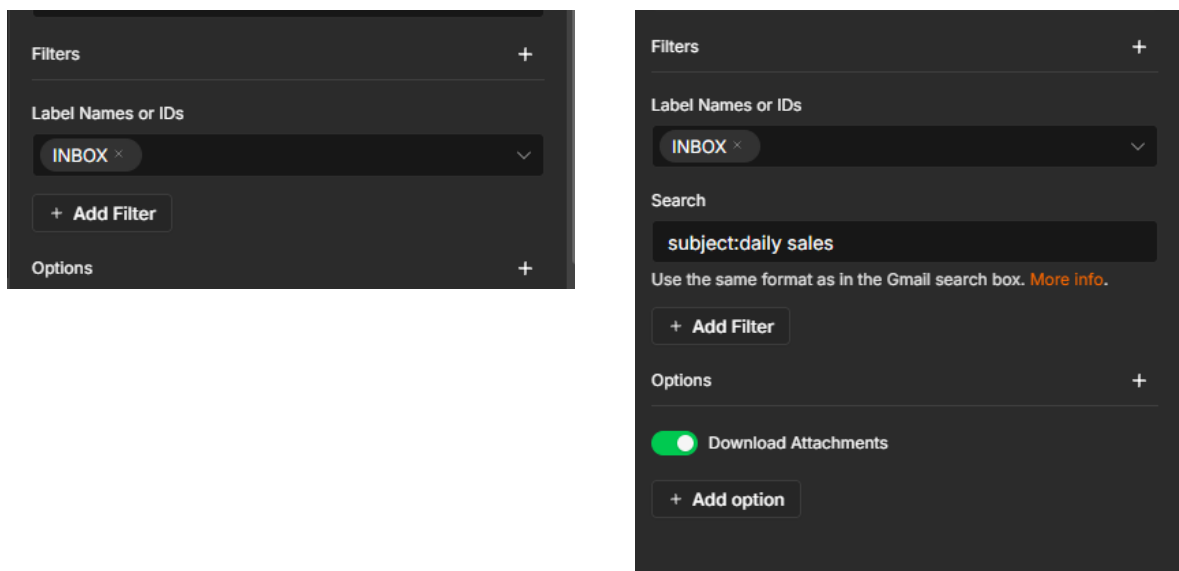


Figure 3: INBOX label filter (left) and subject search filter with Download Attachments (right)

- Click “Fetch Test Event”.
- Add a new node > **Extract from File** > **Extract From CSV**.
- Set **Input Binary Field** to the attachment name (e.g. attachment_0) and click **Execute step** in the right-hand column.

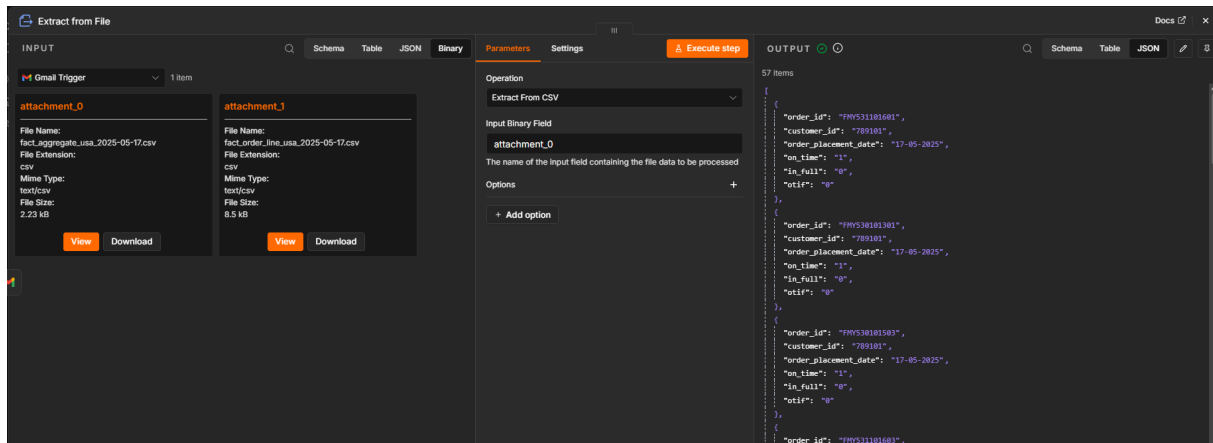


Figure 4: Extract From CSV node – the attachment data appears as JSON

12. The data is now visible as JSON, ready to be inserted into Postgres.
13. Import your CSV files into Supabase and connect the Postgres database to N8N using an **Insert rows in a table** node.

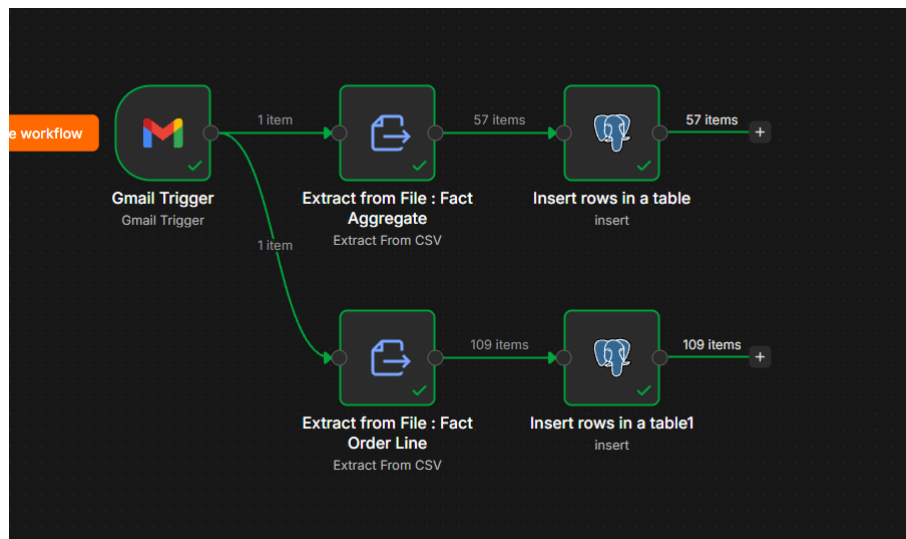


Figure 5: Final workflow – Gmail Trigger > Extract from File > Postgres insert (fact_aggregate: 57 rows, fact_order_line: 109 rows)

4. Analysis Using Quadratic

4.1 Pull the data from Postgres

- In the Quadratic **Schema** panel, click the table you want to pull from. The data loads automatically.
- Repeat for every required table.
- Query for the first 100 rows:

```
SELECT * FROM "public"."dim_customers" LIMIT 100
```

- Query for all rows:

```
SELECT * FROM "public"."dim_customers"
```

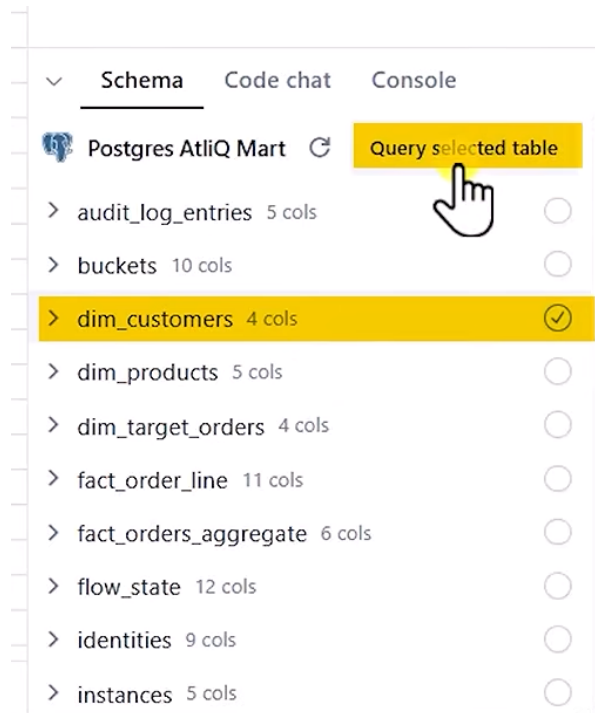


Figure 6: Postgres AtliQ Mart schema in Quadratic – select a table and click “Query selected table”

4.2 Create the dim_date table

As a data analyst, the **dim_date** table is very important: it provides the timeframe and shows which financial quarter each record falls in. As in Excel, create a new sheet and rename it *dim_date* – but in Quadratic a prompt can do it for you:

Prompt: Create a date table that has the dates from 03-01-2025 (March 1st, 2025) to 05-31-2025 (May 31st, 2025).

4.3 Create the exchange_rates table

Currency-conversion rates are needed for more reliable analysis.

Prompt: Create an exchange_rate table for the period from March 1st, 2025, to May 17th, 2025. Use the following API request sample to fetch historical exchange rates:

```
response = requests.get(f'https://openexchangerates.org/api/historical/2024-10-21.json?app_id=<YOUR_APP_ID>&symbols=INR')
```


 Replace the date in the URL with each date in the specified range.

After the output is generated, use **Code chat** to modify the code or change how the values are fetched.

4.4 Data preprocessing and cleaning

Use the following prompt:

Create a Python code that:

1. Loads data from:

- fact_order_line table
- dim_products sheet (skip first row, use second row as headers)
- dim_customers sheet (skip first row, use second row as headers)
- Exchange Rate sheet (skip first row, use second row as headers)

2. Cleans the data:

- Convert product_id and customer_id to numeric
- Strip whitespace from IDs
- Remove rows with NULL IDs
- Convert IDs to integers
- Convert dates to datetime

3. Merges the tables:

- Orders with products using product_id
- Result with customers using customer_id
- Result with exchange rates using order_placement_date

4. Calculates total amounts:

- For USD: $\text{price_USD} \times \text{USD_INR_Rate} \times \text{order_qty}$
- For INR: $\text{price_INR} \times \text{order_qty}$

5. Cleans the final output:

- Drop intermediate columns: price_USD, price_INR, currency, Date, USD_INR_Rate
- Keep only essential columns: order details, IDs, dates, quantities, delivery information and total_amount (in INR)

5. Supply Chain Domain Knowledge

5.1 Basic KPIs

KPI	Level
Total Order Lines	Order line
Line Fill Rate (LFR)	Order line
Volume Fill Rate (VOFR)	Quantity
Total Orders	Order
On Time Delivery % (OT)	Order
In Full Delivery % (IF)	Order
On Time In Full % (OTIF)	Order

5.2 Orders vs Order Lines

Suppose a customer orders 1 keyboard today. The next day he orders 2 different items (a keyboard and a mouse) together. He has placed **2 orders**, but in the sheet each product sits on its own row, giving **3 order lines**.

- An **order** is a unique request placed by a customer on a given date.
- Within an order, a customer may request several items. Each requested item is an **order line**.
- Example: ordering 4 notebooks and 2 pens on Amazon creates one order ID; the notebook and the pen are two order lines.

Date	Order Id	Items	Qty ordered
19/05/2025	order1	Keyboards	5
20/05/2025	order2	Keyboards	10
20/05/2025	order2	Mouse	5

Table 1: Sample orders – Orders = 2, Total Order Lines = 3

5.3 Line Fill Rate vs Volume Fill Rate

Add a column showing whether each line was delivered in full. If a customer orders 5 keyboards and all 5 are delivered, it is a **Yes** (1 in binary); if the line was not delivered in full, it is a **No** (0). With 3 order lines, of which 2 are Yes and 1 is No, the Line Fill Rate is 2/3.

Line Fill Rate = Number of order lines delivered in full ÷ Total order lines

For the Volume Fill Rate we use quantities instead of lines. For example, if total quantity ordered = 20 and quantity delivered = 17:

Volume Fill Rate = Quantity delivered ÷ Total quantity ordered

Date	Order Id	Items	Qty ordered	Qty delivered	Delivered in full?
19/05/2025	order1	Keyboards	5	5	1
20/05/2025	order2	Keyboards	10	7	0

Date	Order Id	Items	Qty ordered	Qty delivered	Delivered in full?
20/05/2025	order2	Mouse	5	5	1
Total			20	17	

Table 2: Sample data with delivery quantities

Metric	Value	Used by
Orders	2	
Order Lines	3	
Total Lines Delivered in Full	2	
Line Fill Rate	66.67%	Supply planners / managers
Volume Fill Rate	85.00%	Supply planners / sales

Table 3: Fill-rate results

- These two are very important metrics for supply planners, supply managers and production managers.
- **Line Fill Rate** tells the planning team how many lines they shipped out of the total lines ordered. It does not consider delivery time.
- **Volume (case) Fill Rate** shows the total quantity the team was able to ship for a customer, per order or over a period.

5.4 On Time and In Full

Expand the table with **Agreed Delivery Date**, **Actual Delivery Date** and a “**Delivered on time?**” column.

- **In Full (IF)** is measured at the **order** level. An order is In Full only when *all* its line items are delivered in full.
- **On Time (OT)** is also measured at the order level. An order is On Time only when *all* its line items are delivered by the agreed date.

Note: In Full is counted by orders, not by order lines. In our example, Order 1 has 1 line and Order 2 has 2 lines. Order 2 was not delivered in full for one product, so In Full Orders = **1** (not 3). Order 2 was still delivered on time, so On Time Orders = **2**.

Date	Order Id	Items	Qty ordered	Qty delivered	Delivered in Full	Agreed delivery	Actual Delivery	Delivered
20/05/2025	order2	Keyboards	10	7	0	21/05/2025	21/05/2025	1
20/05/2025	order2	Mouse	5	5	1	21/05/2025	21/05/2025	1
			20	17				
Orders		2						
Total Lines		3						
Total Lines Delivered		1						
Line Fill Rate		33.33%						
VOFR		85.00%						
In Full Orders (IF)		1	Who uses it?					
On Time Orders (OT)		2						

Figure 7: Sample sheet extended with delivery dates and order-level KPIs (In Full Orders = 1, On Time Orders = 2). Mostly used by warehouse and distribution teams.

5.5 On Time In Full % (OTIF) – very important

- OTIF is measured at the order level and checks whether an order was delivered **both** in full **and** on time, as per the customer's request.
- It matters to every sub-function of the supply chain team.
- An order is OTIF only when every line item is delivered In Full and On Time. It is a hard metric that measures the reliability of an order from the customer's point of view.
- Order 1 was delivered in full and on time → **Yes**. Order 2 was on time but one product was short → **No**.
- If a customer orders 500 different products and 499 arrive in full and on time but one is short, the order scores 0. That is why OTIF is called a very harsh metric.

OTIF % = Orders delivered On Time and In Full ÷ Total orders

Tip: use consolidated data based on orders, not on separate order lines.

5.6 Example Python code (Quadratic)

```
import pandas as pd

# Get order line data
order_lines = q.cells("'fact_order_line'!A2:K24532", first_row_header=True)
order_agg = q.cells("'fact_aggregate'!A2:F13654", first_row_header=True)

# Calculate KPIs
total_order_lines = len(order_lines)
line_fill_rate = (order_lines['In Full'] == '1').mean() * 100
volume_fill_rate = (order_lines['delivery_qty'].sum() /
                    order_lines['order_qty'].sum()) * 100
total_orders = len(order_agg)
on_time_delivery = (order_agg['on_time'] == 1).mean() * 100
in_full_delivery = (order_agg['in_full'] == '1').mean() * 100
otif = (order_agg['otif'] == '1').mean() * 100
```